**1. Dynamic Programming and Value Iteration for Warehouse Robot Navigation.**

A warehouse robot must transport goods from a starting location to a specified destination while avoiding shelves, walls, and other obstacles. The warehouse is represented as a grid, where each cell represents a possible state of the robot.

Apply Dynamic Programming to formulate the warehouse navigation problem as a Markov Decision Process (MDP). Define the states, actions, transition probabilities, and reward function. Develop an optimal policy using Value Iteration and explain how repeated updates of the state-value function help the robot select shorter and safer paths. Discuss how the discount factor and obstacle penalties affect navigation efficiency.

Expected Points

- Define the warehouse states and possible actions.
- Formulate the reward function for reaching the destination and avoiding obstacles. •

  Initialize the value function.
- Apply the Bellman optimality equation.
- Iteratively update state values until convergence.
- Extract the optimal policy from the final value function.
- Explain how Value Iteration improves path selection and reduces unnecessary movements.

Key Concept

Environment → States → Actions → Rewards → Value Iteration → Optimal Value Function → Optimal Navigation Policy

**2. Monte Carlo Control for Customer Recommendation System**

A recommendation system interacts with customers over multiple sessions. During each session, the system recommends products, movies, or services and receives feedback based on customer interactions such as clicks, purchases, ratings, or skips.

Apply Monte Carlo Control to estimate the action-value function from complete customer interaction episodes. Explain how the system records the states, recommendations, and rewards during each session and updates the action-value estimates after an episode is completed. Describe how an ε-greedy policy can balance exploration and exploitation and explain how repeated customer experiences improve the quality of future recommendations.

Expected Points

- Define customer states and recommendation actions.
- Define rewards based on customer interactions.
- Generate complete recommendation episodes.
- Calculate the return from observed rewards.
- Update the action-value function using sampled experiences.
- Use an ε-greedy strategy for exploration and exploitation.
- Improve recommendations based on accumulated customer experience.

Example

Customer Profile → Recommendation → Customer Response → Reward → Complete Session → Monte Carlo Update → Improved Recommendation

Key Concept

Monte Carlo Control learns from complete episodes of experience rather than updating values after every individual action.

**3. SARSA for Adaptive Delivery Robot Navigation**

A delivery robot operates in an uncertain environment where obstacles, people, and other moving objects may change its route. The robot must deliver packages efficiently while adapting its behavior when the environment changes.

Apply the SARSA algorithm to update the action-value function of the delivery robot. Define the states, actions, rewards, and ε-greedy action-selection strategy. Explain how SARSA

updates the Q-value using the current state-action pair and the next state-action pair. Analyze how the learned policy can adapt when obstacles or environmental conditions change.

SARSA Update

The action-value function can be updated using:

$$Q(S,A) \leftarrow Q(S,A) + \alpha\,[R + \gamma Q(S',A') - Q(S,A)]$$

where:

- $S$ = Current state
- $A$ = Current action
- $R$ = Reward received
- $S'$ = Next state
- $A'$ = Next action
- $\alpha$ = Learning rate
- $\gamma$ = Discount factor

Expected Points

- Define the delivery environment.
- Initialize the Q-table.
- Select an action using ε-greedy policy.
- Execute the action and observe the reward.
- Select the next action.
- Apply the SARSA update rule.
- Repeat until the delivery task is completed.
- Explain how repeated interaction enables adaptation.

Key Concept

SARSA is an on-policy Temporal Difference learning algorithm because it learns the value of

the policy that the robot is actually following.

**4. REINFORCE for Robotic Arm Object Manipulation**

A robotic arm must perform continuous object manipulation tasks such as grasping, lifting, moving, and placing objects. The robot operates in an environment where small variations in object position and movement can affect task completion.

Apply the REINFORCE policy-gradient algorithm to develop a policy that maximizes the probability of successfully completing the manipulation task. Define the state representation, continuous action space, reward function, and policy network. Explain how complete episodes are used to calculate returns and update the policy parameters. Discuss how repeated training improves manipulation accuracy.

Expected Points

- Define the robotic arm state, such as joint angles, object position, and velocity. •

  Define continuous actions such as joint movements.

- Design a parameterized policy network.
- Execute actions according to the learned policy.
- Calculate rewards for successful grasping and placement.
- Calculate the return for each episode.
- Update policy parameters using the policy-gradient rule.

REINFORCE Principle

The policy is updated in the direction that increases the probability of actions that produced higher returns:

$\theta \leftarrow \theta + \alpha\, G_t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t)$

where:

- $\theta$ = Policy parameters
- $\alpha$ = Learning rate
- $G_t$ = Return from time step $t$

• $\pi\theta(A_t|S_t)$ = Probability of selecting action $A_t$ in state $S_t$

Key Concept

State → Policy Network → Continuous Action → Environment → Reward → Return → Policy Update

**5. Q-Learning vs DQN for Autonomous Vehicle Learning**

An autonomous vehicle must learn to make driving decisions such as accelerating, braking, changing lanes, and maintaining direction in a complex environment containing traffic, pedestrians, intersections, and unexpected obstacles.

Compare the application of traditional Q-Learning and Deep Q-Networks (DQN) for autonomous vehicle control. Analyze their differences in learning performance, convergence speed, memory requirements, stability, and scalability when the state space becomes large or continuous. Explain how DQN uses a neural network to approximate the Q-function and discuss the role of experience replay and a target network in improving learning stability.

Comparison

| Factor | Q-Learning | DQN |
|---|---|---|
| State representation | | Can handle high-dimensional |
| discrete Q-function | Q-table | states Neural network |
| Memory | Q-table | Replay memory + neural |
| Large state space In | Difficult | network More scalable |
| input Not suitable d | | Suitable |
| Training complexity | | High |
| Convergence | | problems Can require more |
| Scalability | Limited | training Better for complex |

| | | |
|---|---|---|
| | | environments |

Q-Learning Update

$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max Q(S',A') - Q(S,A)]$

DQN

DQN replaces the Q-table with a deep neural network that estimates Q-values for possible driving actions.

Two important mechanisms are:

1. Experience Replay – stores previous experiences and randomly samples them for training.
2. Target Network – provides a more stable target during Q-value updates.

Expected Analysis

For a small and simple driving environment, Q-Learning can be easier and faster to implement. However, for realistic autonomous driving with large state spaces, camera images, LiDAR information, traffic conditions, and numerous possible states, DQN is more scalable because a neural network can approximate the Q-function instead of maintaining a huge Q-table.

Comparison

| Factor | Q-Learning | DQN |
|---|---|---|
| State representation | | Can handle high-dimensional |
| discrete Q-function | Q-table | states Neural network |
| Memory | Q-table | Replay memory + neural |
| Large state space In | Difficult | network More scalable |
| | | Suitable |

| | | |
|---|---|---|
| input Not suitable d<br>Training complexity<br>Convergence<br>Scalability | <br><br>Limited | High<br>problems Can require more<br>training Better for complex<br>environments |

Q-Learning Update

$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max Q(S',A') - Q(S,A)]$

DQN

DQN replaces the Q-table with a deep neural network that estimates Q-values for possible driving actions.

Two important mechanisms are:

RUBRICS FOR CALCULATION

| Criteria | Weightag e | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Conceptual Understanding | 25 | Demonstrates deep understandin g of concepts with complete clarity | Explains concepts correctly with minor gaps | Partial understanding with errors or omissions | Unable to explain basic concepts |

| | | | | | |
|---|---|---|---|---|---|
| Application | 25 | Effectively applies concepts to new situations; solves problems logically | Applies concepts with minor errors | Limited ability to apply concepts; requires guidance | Unable to apply concepts effectively |
| Analytical Thinking | 20 | Critically analyzes scenarios with strong | Provides reasonable analysis with some justification | Superficial analysis with weak reasoning | No analytical reasoning demonstr ate d |
| | | justificatio n and reasoning | | | |
| Communica tio n | 15 | Communica t es ideas clearly, confidently, and with appropriate terminology | Communica t es clearly but with minor hesitation | Communicat io n lacks clarity or structure | Unable to communic at e ideas effectively |

| | | | | | |
|---|---|---|---|---|---|
| Response to Questions | 15 | Responds accurately and confidently, extending answers with depth | Responds correctly but with limited depth | Responds partially; requires prompting | Unable to respond to questions effectively |